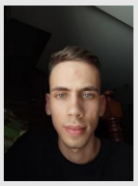
Universidade do Minho
Licenciatura em Engenharia Informática

Unidade Curricular de Programação Orientada aos Objetos

Ano Lectivo de 2025/2026

A111753
João Pedro Cunha Cerqueira



A1122032
Rafael Figueiras Esteves



A110067
Nuno Henrique Dias Pereira



Relatório

DomusControl

Programação Orientada a Objetos | 2025/2026

Resumo

O projeto DomusControl implementa um sistema de gestão de *Smart Homes* em Java, com foco no paradigma de programação orientada a objetos, controlo de acessos por casa, operação de dispositivos, automações condicionais, escalonamentos temporais, cenários de utilização, sugestões inteligentes de automação e persistência de estado em formato binário.

A arquitetura centraliza a lógica de orquestração na classe `DomusControl`, separando o domínio por pacotes (casas, dispositivos, utilizadores, automações, escalonamentos, cenários, sugestões e interface textual). O sistema suporta estatísticas de consumo e uso de dispositivos, incluindo tops de dispositivos e divisões, e oferece recomendações proativas baseadas na análise de padrões de utilização.

Objetivos do Projeto

- Modelar os principais elementos de uma casa inteligente com encapsulamento e reutilização.
- Suportar múltiplas casas e múltiplos utilizadores com diferentes níveis de permissão.
- Permitir automação por regras (condição + ação) e por agendas (instante de execução).
- Implementar cenários pré-definidos e cenários personalizados.
- Gerar sugestões inteligentes de automação com base em padrões de uso.
- Disponibilizar interface textual para operações de gestão.
- Garantir persistência e recuperação de estado.

Descrição da Arquitetura e Decisões de Projeto

Padrão Fachada — Centralização no DomusControl

A classe `DomusControl` implementa o padrão *Facade*: é o ponto de entrada único para toda a lógica do sistema. A `TextUI` nunca manipula diretamente objetos como `Casa` ou `Dispositivo` — todas as operações passam pelo controlador, que valida permissões, regista estatísticas e garante consistência. Esta separação permite trocar a interface (por exemplo, de textual para gráfica) sem alterar a lógica de negócio.

As estatísticas de uso (ativações, horas de uso, instante de ligação) foram mantidas no `DomusControl` e não nas classes `Casa` ou `Dispositivo`, para não poluir o domínio com responsabilidades de contabilização — respeitando o princípio de responsabilidade única.

Escolha de Estruturas de Dados

Optámos por `Map<String, ...>` com chaves UUID em vez de listas para todas as entidades principais (casas, utilizadores, dispositivos, divisões, automações, cenários). Esta decisão garante acesso em tempo constante (*O(1)*) por identificador, operação fundamental num sistema onde se consultam e manipulam entidades individuais com frequência. Os `UUID` são gerados automaticamente no construtor e não possuem *setter* público, garantindo identidades estáveis e únicas ao longo do ciclo de vida do sistema.

A exceção é a lista de `Escalonamento`, que usa `List` por ser percorrida sequencialmente na execução pendente, sem necessidade de acesso aleatório por chave.

Classe Abstrata vs. Interface para Dispositivos

A classe `Dispositivo` foi declarada como **abstrata** e não como interface porque todos os dispositivos partilham estado e comportamento comuns (atributos de identificação, consumo, estado ligado/desligado e respetivos métodos). Uma interface obrigaria cada subclasse a reimplementar esta lógica, violando o princípio DRY. O único método abstrato é `clone()`, forçando cada subclasse a fornecer a sua própria cópia profunda.

```
Dispositivo (abstrata)
├── Rele
├── Lampada
├── Cortina
├── PortaoGaragem
├── ColunaSom
├── ArCondicionado
├── FechaduraInteligente
├── SistemaDeRega
└── SmartTV
```

Listagem 1: Hierarquia de herança dos dispositivos

Interfaces Funcionais — Padrão Strategy

As interfaces `CondicaoAutomacao` e `AcaoAutomacao` foram desenhadas como interfaces funcionais (um único método abstrato), permitindo definir regras com expressões *lambda*. Esta decisão segue o padrão *Strategy*: a `Automacao` delega a lógica para objetos que implementam estas interfaces, sem conhecer o comportamento concreto.

A mesma interface `AcaoAutomacao` é reutilizada em três contextos distintos — automações condicionais, escalonamentos temporais e cenários manuais — demonstrando a extensibilidade e coerência da abstração escolhida.

Controlo de Acessos Bidirecional

A associação entre casas e utilizadores é mantida em ambos os lados: a `Casa` guarda email ⟶ permissão e o `Utilizador` guarda idCasa ⟶ permissão. Esta redundância controlada permite consultas eficientes nas duas direções (casas de um utilizador, ou utilizadores de uma casa) sem travessias exaustivas, sacrificando ligeiramente a normalização em favor da performance.

Encapsulamento Defensivo

- **Clonagem nos getters:** todos os *getters* que devolvem coleções ou objetos de domínio devolvem cópias profundas, nunca referências internas. Isto previne *aliasing* — modificações externas não corrompem o estado interno.
- **Construtores de cópia:** todas as classes implementam um construtor de cópia que serve de base ao `clone()`, uniformizando a criação de cópias.
- **Validação fail-fast:** os *setters* e construtores validam os dados de entrada e lançam `IllegalArgumentException` com mensagem descritiva quando inválidos, preferindo falhar ruidosamente a aceitar estado corrompido.
- **Constantes defensivas:** a classe `Automacao` inclui `CONDICAO_FALSA` e `ACAO_VAZIA` como *defaults* seguros para instâncias não configuradas, prevenindo `NullPointerException`.

Operações Genéricas com Consumer

O método `operarSobreDispositivo` aceita um `Consumer<Dispositivo>`, permitindo à interface executar operações específicas por tipo (ex.: alterar volume de uma `ColunaSom`) sem que o controlador precise de conhecer o tipo concreto — combinando programação funcional com polimorfismo.

Persistência por Serialização
Todas as classes de domínio implementam `Serializable`, permitindo gravar e restaurar o estado completo do sistema com `ObjectOutputStream`/`ObjectInputStream`. Esta abordagem foi escolhida pela sua simplicidade e por garantir a preservação integral do grafo de objetos, incluindo relações entre entidades.

Diagrama de Classes
Dada a elevada densidade de informação e a complexidade das relações entre as diversas entidades do sistema, optou-se por subdividir a representação em módulos lógicos, garantindo assim a clareza e a legibilidade de cada componente.

Ressalva-se que os diagramas foram exportados em formato **.svg**, o que permite uma exploração detalhada através de *zoom* sem qualquer perda de definição, assegurando a consulta precisa de todos os atributos e métodos, independentemente da escala de visualização. Os diagramas referidos estão no fim do relatório.

Organização por pacotes
- `controlador`: coordenação global do sistema (`DomusControl`).
- `casas`: modelo de casa, divisões e tipo de acesso.
- `dispositivos`: hierarquia de dispositivos controláveis.
- `utilizadores`: entidade de utilizador e acessos.
- `automacoes`: regras de automação com condições e ações.
- `escalonamentos`: tarefas agendadas para execução futura.
- `cenarios`: agrupamento de ações para modos de funcionamento.
- `sugestoes`: recomendações inteligentes baseadas em padrões de uso.
- `UI`: interface textual e menus.
- `testes`: testes unitários automatizados (JUnit 5).

Classe central
A classe `DomusControl` atua como fachada da aplicação:

- gere utilizadores e casas;
- aplica regras de acesso;
- regista ativações e tempo de uso de dispositivos;
- executa automações e escalonamentos;
- aplica cenários;
- gera sugestões inteligentes de automação;
- calcula estatísticas do sistema;
- grava e lê estado binário.

Modelo de Domínio

Utilizadores e acessos
A classe `Utilizador` representa as pessoas que interagem com o sistema. Os seus atributos principais são:
- `nome` (String) e `email` (String): dados de identificação;
- `acessosCasas` (`Map<String, TipoAcessoCasa>`): mapeia o ID da casa ao nível de permissão.

O `TipoAcessoCasa` é um enumerado (`Enum`) que distingue dois tipos de permissão:
- `ADMINISTRADOR`: pode executar operações administrativas (ex.: adicionar divisão/dispositivo).
- `UTILIZADOR`: pode apenas operar dispositivos e funcionalidades autorizadas.

Casas e divisões
A classe `Casa` centraliza o estado de uma residência e agrega os seguintes atributos:

- O identificador `idCasa` (String formatada como UUID);
- Atributos de metadados: `nomeProprietario` (String), `codigoPostal` (int), `fornecedorEnergia` (String);
- `acessosUtilizadores` (`Map<String, TipoAcessoCasa>`): dita quem pode aceder e com que permissão;
- `dispositivos` (`Map<String, Dispositivo>`): dicionário de todos os equipamentos da habitação;
- `divisoes` (`Map<String, Divisao>`): zonas/divisões criadas na casa;
- `automacoes` (`Map<String, Automacao>`): regras independentes registadas.

Por sua vez, a classe `Divisao` armazena o `nome` e uma lista de referências `idDispositivos` (`List<String>`). Foi decidido guardar apenas os IDs dos dispositivos nas divisões (e não os objetos em si) para evitar duplicação de referências de memória e garantir uma única fonte de verdade no mapa de dispositivos da Casa.

Dispositivos

A classe abstrata `Dispositivo` define os atributos e comportamentos comuns a todos os equipamentos:

- `id` (String), `marca` (String) e `modelo` (String);
- `consumoPorHora` (double): energia consumida quando o equipamento liga;
- `estado` (boolean): dita se está ligado ou desligado.

A partir desta classe base, estendemos as classes concretas recorrendo ao polimorfismo e herança:

- `Rele`: dispositivo de corte simples (ligar/desligar).
- `Lampada`: acrescenta atributos de `intensidade` (int), `suportaCor` (boolean) e `temperaturaCor` (int).
- `Cortina`: acrescenta a `percentagemAbertura` (int).
- `PortaoGaragem`: estrutura semelhante à cortina, gere `percentagemAbertura` (int).
- `ColunaSom`: acrescenta o atributo de `volume` (int).
- `ArCondicionado`: acrescenta os atributos `velocidade` (int), `modo` (String) e `temperatura` (double).
- `FechaduraInteligente`: gere a segurança com `estadoTranca` (boolean) e `codigoAcesso` (String), com métodos para destrancar via PIN.
- `SistemaDeRega`: adiciona controlo de `percentagemIrrigacao` (int), `duracao` (int) e `zona` (String).
- `SmartTV`: introduz atributos de controlo de media como `volume` (int), `canalAtual` (int) e `fonteEntrada` (String).

Funcionalidades Implementadas

Gestão base

- criação/remoção/listagem de utilizadores;
- criação/remoção/listagem de casas;
- atribuição de acessos a casas;
- criação de divisões;
- associação de dispositivos a divisões.

Controlo de dispositivos

- ligar/desligar por id;
- ligar/desligar por divisão;
- operações específicas por tipo de dispositivo (ex.: intensidade da lâmpada, volume da coluna);
- operação genérica via `Consumer<Dispositivo>`, permitindo executar qualquer ação polimórfica sobre um dispositivo sem expor o modelo interno.

Automações

Uma automação contém:

- condição (`CondicaoAutomacao`);

- ação (`AcaoAutomacao`);
- estado ativa/inativa.

A execução ocorre quando a condição é verdadeira, permitindo regras declarativas sobre o estado da casa. As interfaces `CondicaoAutomacao` e `AcaoAutomacao` estendem `Serializable`, recebendo uma `Casa` como argumento — o que permite definir condições e ações arbitrárias via expressões *lambda*. A classe `Automacao` inclui constantes defensivas (`CONDICAO_FALSA` e `ACAO_VAZIA`) como *defaults* seguros para instâncias não configuradas.

Escalonamentos

Cada escalonamento define:

- nome;
- casa alvo;
- instante de execução;
- ação;
- indicador de execução.

O método de execução pendente processa eventos cujo instante já foi atingido pelo tempo atual do sistema.

Cenários

O sistema suporta:

- cenários base pré-carregados (Sair de Casa, Jantar com Amigos, Deitar, Acordar);
- criação de cenários personalizados como listas de ações;
- aplicação de cenário por id e casa.

Estatísticas

- casa com maior consumo atual;
- top 3 dispositivos mais usados (global);
- top 3 dispositivos por casa (por tempo e por ativações);
- top 3 divisões com mais dispositivos.

Sugestões Inteligentes de Automação

O sistema implementa um mecanismo de recomendação inteligente de automações, assente na análise de padrões de uso dos dispositivos. Esta funcionalidade destaca-se por oferecer ao utilizador sugestões proativas, transformando dados históricos em ações concretas.

Arquitetura

A funcionalidade assenta em duas classes:

- `Sugestao` (pacote `sugestoes`): encapsula uma recomendação individual, contendo:
  - `idCasa` e `idDispositivo`: contexto da sugestão;
  - `nomeDivisao`: localização do dispositivo;
  - `tipo` (String): categoria da sugestão — `AUTOMACAO`, `ESCALONAMENTO` ou `EFICIENCIA`;
  - `descricao`: texto explicativo da recomendação;
  - `ativacoes` e `horasUso`: métricas que justificam a sugestão.
- `DomusControl.gerarSugestoes()`: método que percorre todas as casas e dispositivos, cruzando estatísticas de uso com limiares configuráveis para produzir sugestões relevantes.

Tipos de Sugestão

O sistema gera três tipos de sugestão, cada uma acionada por um limiar diferente:

Tipo	Limiar	Descrição
`AUTOMACAO`	≥ 5 ativações	Dispositivos ligados/desligados repetidamente. Sugere criar uma automação por regra.
`ESCALONAMENTO`	≥ 10 horas de uso	Dispositivos com uso prolongado. Sugere criar um escalonamento temporal.
`EFICIENCIA`	Ligado com consumo ≥ 50 Wh	Dispositivos com consumo elevado ativos. Alerta para possível desperdício energético.

Exemplo de saída

```
=== Sugestões Inteligentes (2) ===
1. [AUTOMACAO] Casa 3f8a... | Dispositivo RELE-1 (Sala) | 7 ativacoes, 4.0h uso |
   Dispositivo ligado/desligado 7x. Considere criar uma automacao para o ativar
automaticamente.
2. [ESCALONAMENTO] Casa 3f8a... | Dispositivo LAMP-1 (Quarto) | 3 ativacoes, 12.5h uso |
   Dispositivo com 12.5h de uso. Considere agendar ligar/desligar automatico.
```

Justificação de Projeto

A ordenação das sugestões é feita por número de ativações decrescente, priorizando os dispositivos com mais interação manual. O método auxiliar `encontrarDivisaoDoDispositivo` resolve a associação inversa entre dispositivo e divisão, percorrendo as divisões da casa para localizar a que contém o dispositivo alvo.

Persistência

Persistência binária com serialização de `DomusControl`:

- `guardarEstadoBinario(caminho)`
- `lerEstadoBinario(caminho)`

Interface de Utilizador (TextUI)

A interface textual baseia-se num sistema de menus interativos estruturados de modo hierárquico pela classe auxiliar `NewMenu` (fornecida no projeto). Os seus menus primários compreendem:

- Gestão de utilizadores e Gestão de casas;
- Cenários (listar, criar personalizado, remover);
- Operação de casa específica (divisões, dispositivos, automações, escalonamentos e cenários);
- Avanço temporal no sistema e apresentação de estatísticas;
- Sugestões inteligentes de automação;
- Rotinas de persistência (gravar/carregar estado).

Ilustração das Funcionalidades

Para demonstrar as funcionalidades implementadas em ação, apresentamos em seguida dois fluxos típicos do utilizador em interação com a UI:

1. Inicialização de um Utilizador, Casa e Dispositivo:

```
1 - Gestão de Utilizadores // (Seleciona '1' - 'Adicionar Utilizador' -> "Email:
ana@email.pt")
2 - Gestão de Casas // (Cria casa com o email 'ana@email.pt' e Fornecedor 'EDP')
    4 - Gerir Casa Específica // (Introduz o ID da casa gerada e o seu email)
        3 - Adicionar Dispositivo // (Seleciona tipo '1 - Lâmpada', com consumo '15' Wh)
        5 - Ligar/Desligar Dispositivo // (Introduz o ID da lâmpada -> Saída:
"Dispositivo ligado.")
```

2. Aplicação de um Cenário predefinido e verificação dos Estados:

```
(Dentro de "Gerir Casa Específica")
9 - Aplicar Cenário
    Cenario{id='...', nome='Jantar com Amigos', descricao='Liga Sala e Cozinha, desliga
o resto'}
Opção: (Introduz o ID correspondente ao "Jantar com Amigos")
Saída: "Cenário aplicado com sucesso."

4 - Listar Dispositivos
Saída: "Dispositivo [ID=L1, Marca=Philips, ... Estado=Ligado]" (Na Sala/Cozinha)
Saída: "Dispositivo [ID=L2, Marca=IKEA, ... Estado=Desligado]" (No Quarto)
```

3. Visualização de Sugestões Inteligentes:

```
6 - Sugestões Inteligentes
Saída:
  === Sugestões Inteligentes (2) ===
  1. [AUTOMACAO] Casa 3f8a... | Dispositivo RELE-1 (Sala) | 7 ativacoes, 4.0h uso |
     Dispositivo ligado/desligado 7x. Considere criar uma automacao para o ativar
automaticamente.
  2. [EFICIENCIA] Casa 3f8a... | Dispositivo LAMP-1 (Quarto) | 3 ativacoes, 2.0h uso |
     Dispositivo ligado com consumo elevado (50.0 Wh). Considere desliga-lo quando nao
estiver em uso.
```

Esta abordagem modular da interface permite a validação dos requisitos e a simulação de contextos funcionais robustos, sem precisar de depender de uma GUI gráfica.

Validação e Testes

Teste integrado

O ficheiro `TesteRequisitosBaseGestao` cobre, de forma integrada:

- criação de utilizadores e casas;
- controlo de acessos;
- operação de dispositivos por id e por divisão;
- estatísticas de uso e consumo;
- persistência e carregamento de estado;
- execução de 5 automações;
- execução de 7 escalonamentos;
- aplicação dos 4 cenários base.

Este teste funciona como prova de conformidade com os requisitos nucleares da base de gestão.

Testes unitários automatizados

Para além do teste integrado, o projeto inclui uma *suite* de testes unitários JUnit 5 no pacote `testes`, organizada por classe:

`CasaTest` — 10 testes que cobrem:

- controlo de acessos (administrador pode criar divisão; utilizador não pode; utilizador sem acesso é bloqueado);
- gestão de dispositivos (adicionar cria divisão se necessário; ligar/desligar individual e por divisão);
- polimorfismo (ligar todos os subtipos de `Dispositivo` via referência polimórfica);
- encapsulamento (`getDispositivos()` devolve clone — modificar a cópia não altera o original);
- `equals`/`clone` (reflexivo, simétrico, transitivo; clone é independente do original).

`DomusControlTest` — 14 testes que cobrem:

- CRUD de entidades (criar utilizador com sucesso e duplicado; criar casa com email inexistente; remoções);
- estatísticas (consumo zero sem dispositivos ligados; consumo atualizado ao ligar; ativações contabilizadas; horas de uso calculadas);
- persistência binária (gravar e carregar preserva estado completo);
- automações (executa quando condição verdadeira; não executa quando falsa);
- escalonamentos (executa no instante correto; não reexecuta);
- cenários (cenário base desliga tudo; cenário personalizado aplica ações);
- sugestões inteligentes (gera sugestão para dispositivos muito usados; não gera para pouco usados).

Decisões de Projeto

- Uso de mapas (`Map`) para acesso eficiente por identificador.
- Uso de `UUID` para identificadores únicos de casas, automações, escalonamentos e cenários.
- Clonagem defensiva em vários *getters* e construtores de cópia para reduzir *aliasing*.
- *Tratamento Robusto de Exceções:* Utilização exaustiva de classes como `IllegalArgumentException` (ex: configurações mal introduzidas na UI ou ids repetidos) e `IllegalStateException` (ex: bloqueio de acessos por utilizadores sem credenciais), promovendo salvaguardas claras do estado invariável do sistema.
- Uso de `Consumer<Dispositivo>` para operações genéricas sobre dispositivos, tirando partido de programação funcional e polimorfismo em simultâneo.

Conclusão

O projeto atinge os objetivos principais de modelação e gestão de *Smart Homes*, demonstrando a aplicação madura dos conceitos de Programação Orientada aos Objetos, nomeadamente encapsulamento, centralização de responsabilidades, polimorfismo e reutilização. O sistema de sugestões inteligentes constitui uma camada de valor acrescentado que transforma dados históricos de utilização em recomendações acionáveis, diferenciando o projeto de uma implementação puramente funcional. A validação através de testes unitários JUnit 5 e de testes integrados garante a conformidade com os requisitos. A estrutura modular concebida facilita assim a sua futura extensão, tornando a aplicação escalável a diferentes topologias de automação doméstica.

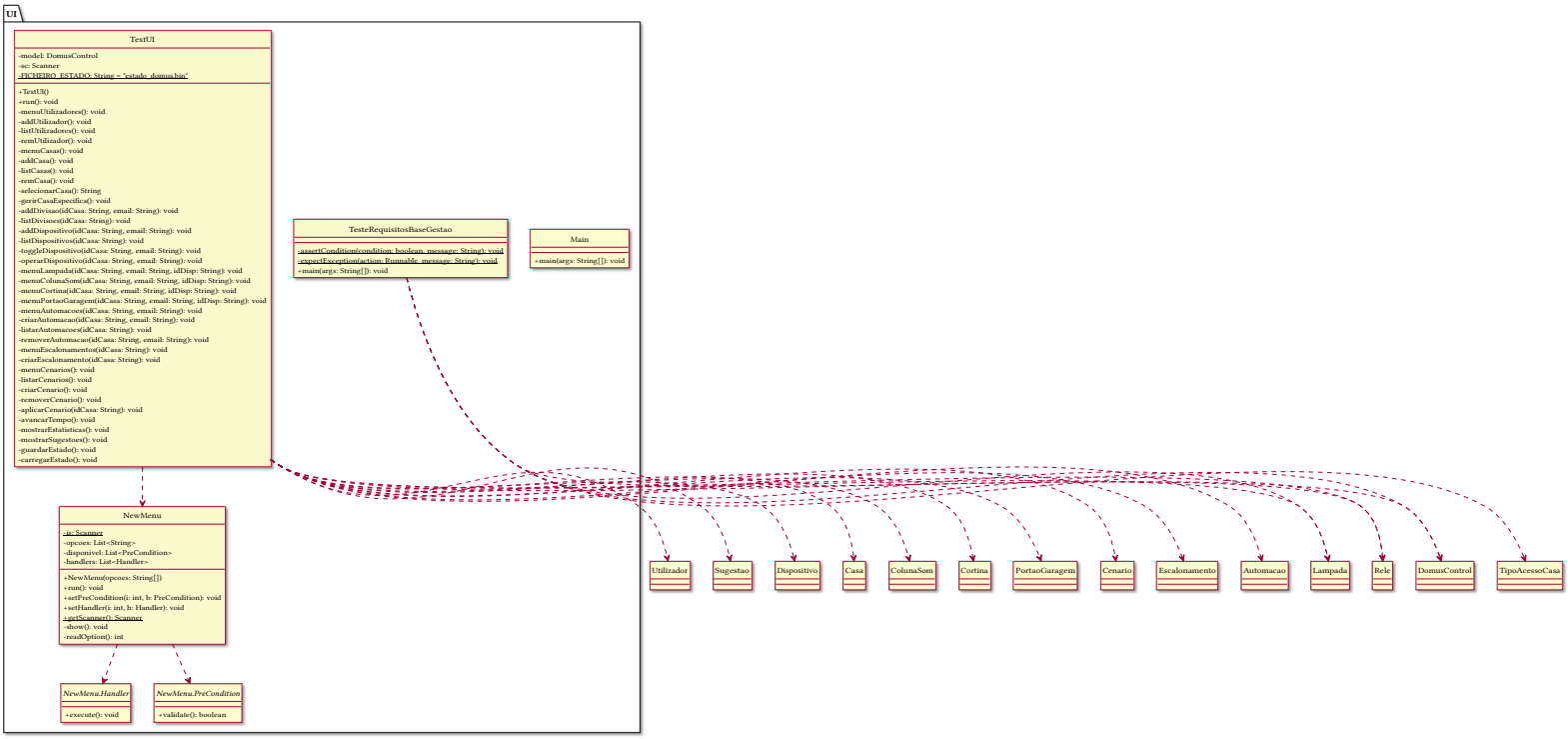


Figura 1: Diagrama de classes - Interface (UI)

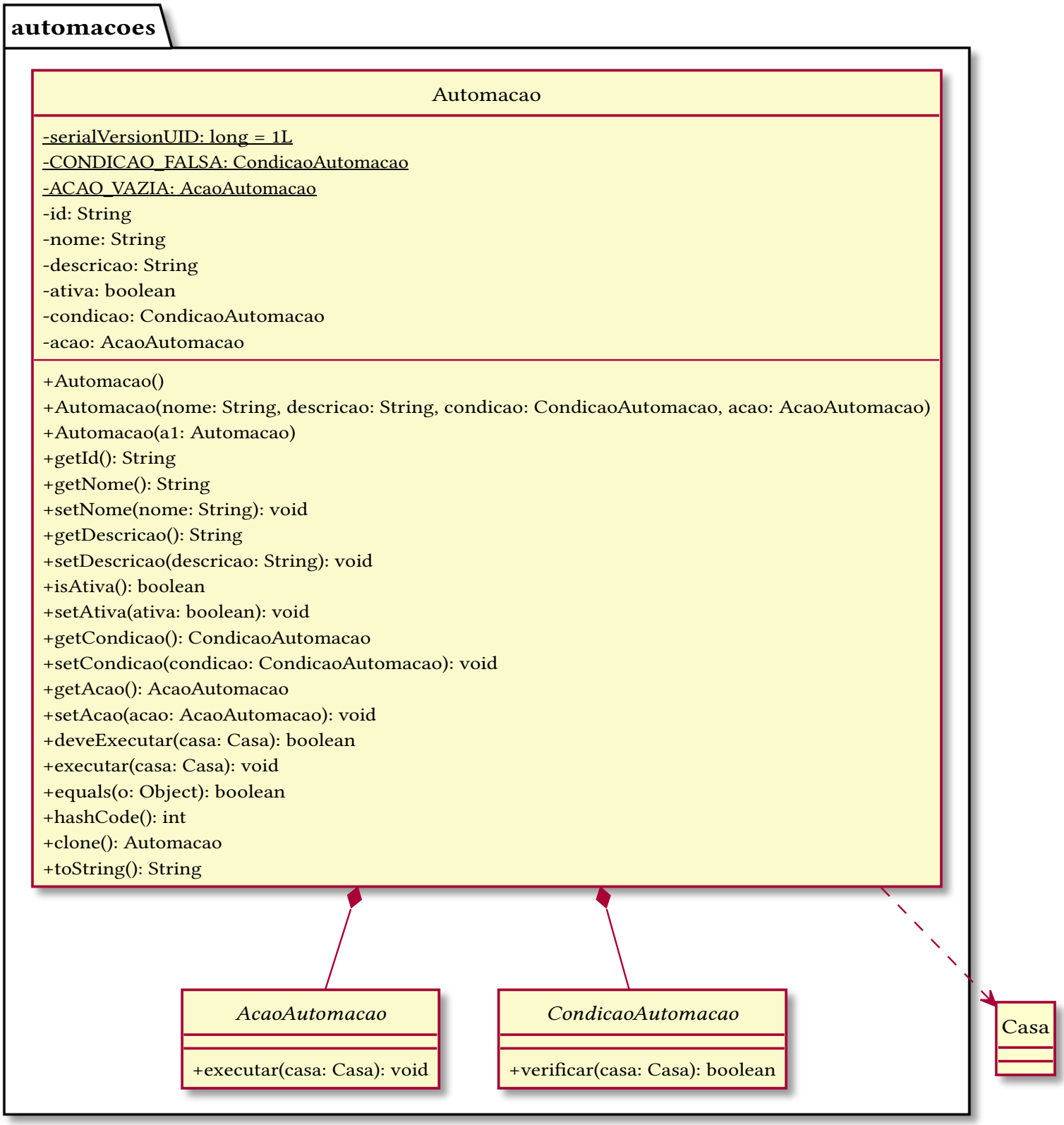


Figura 2: Diagrama de classes - Automacoes

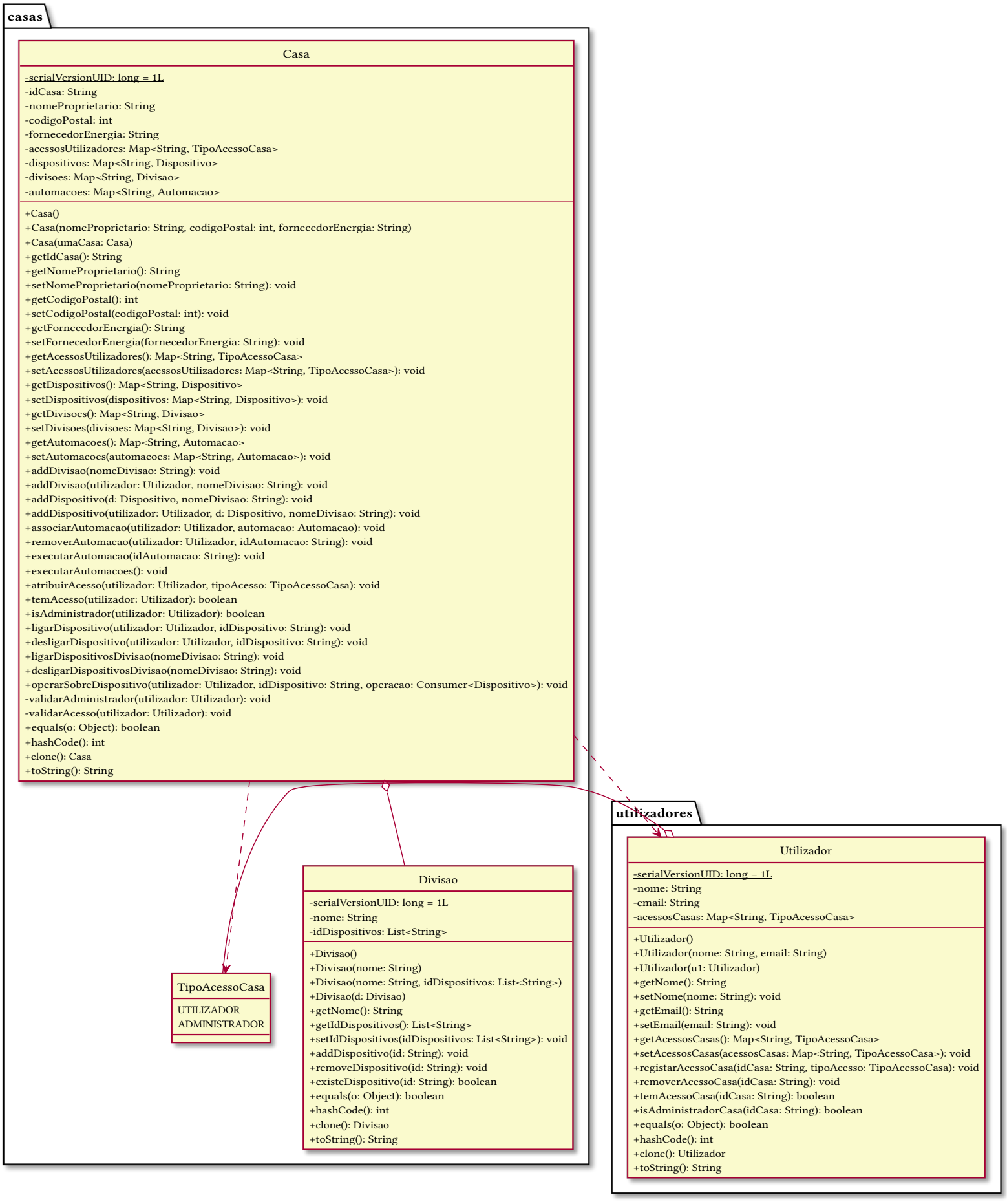


Figura 3: Diagrama de classes - Casas e Utilizadores

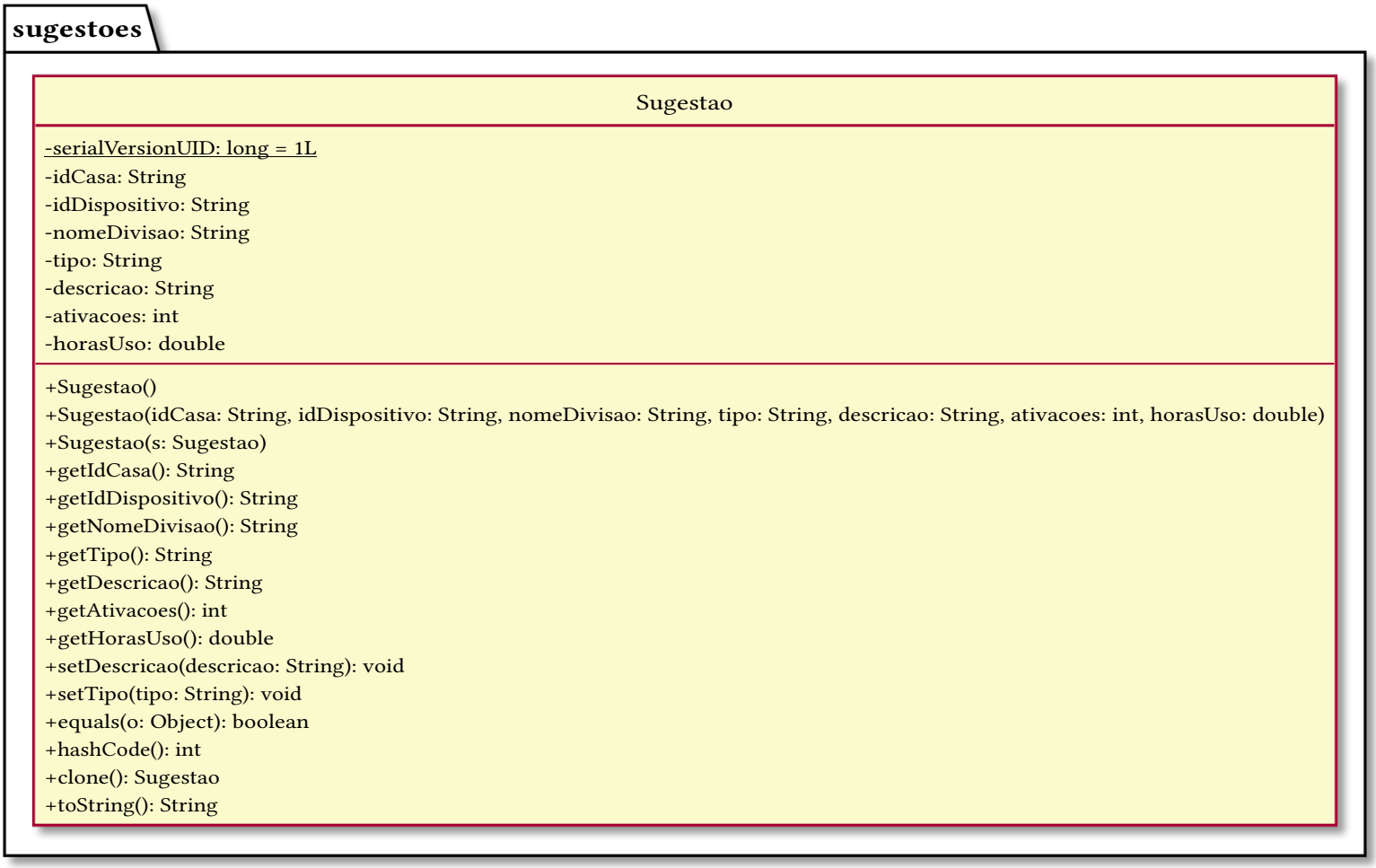


Figura 4: Diagrama de classes - Sugestões

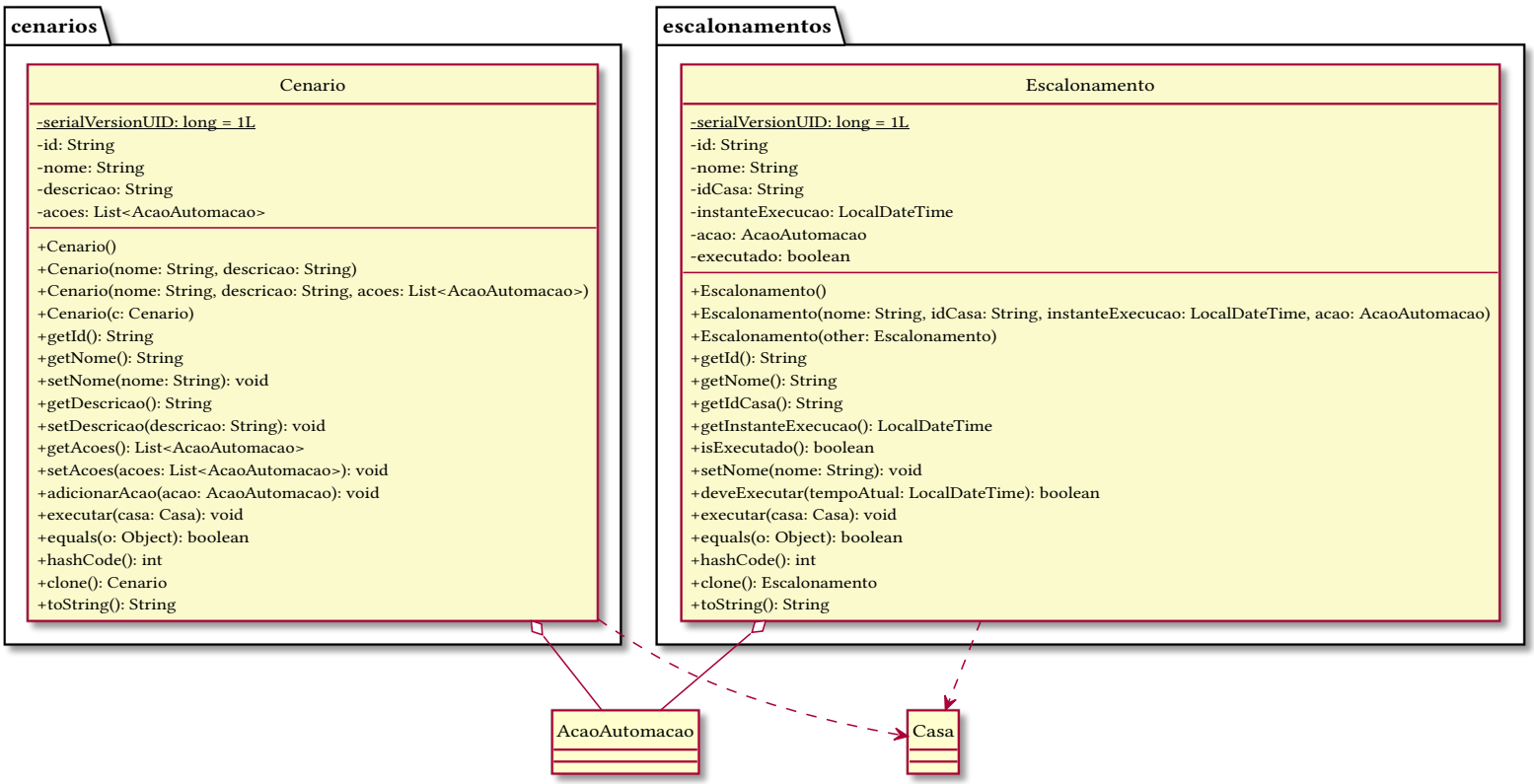


Figura 5: Diagrama de classes - Cenarios e Escalonamentos



Figura 6: Diagrama de classes - Controlador

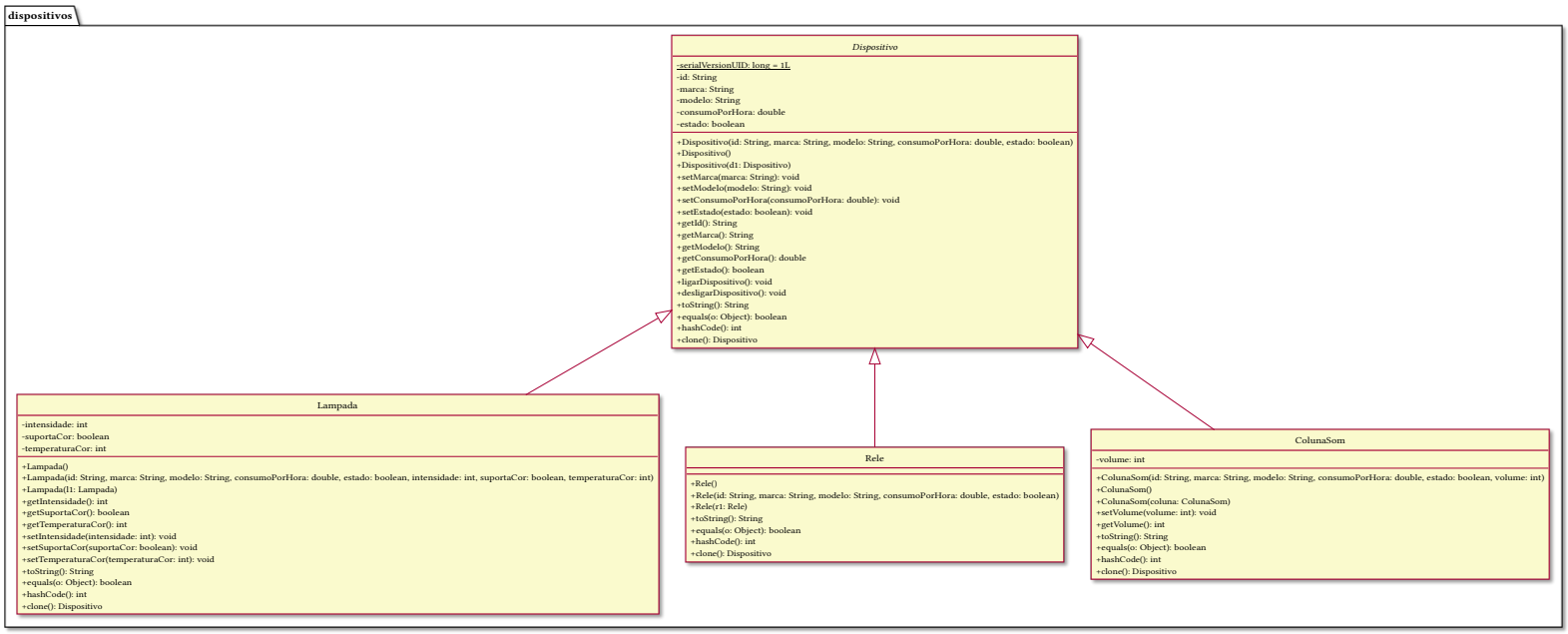


Figura 7: Diagrama de classes - Dispositivos (Parte 1)

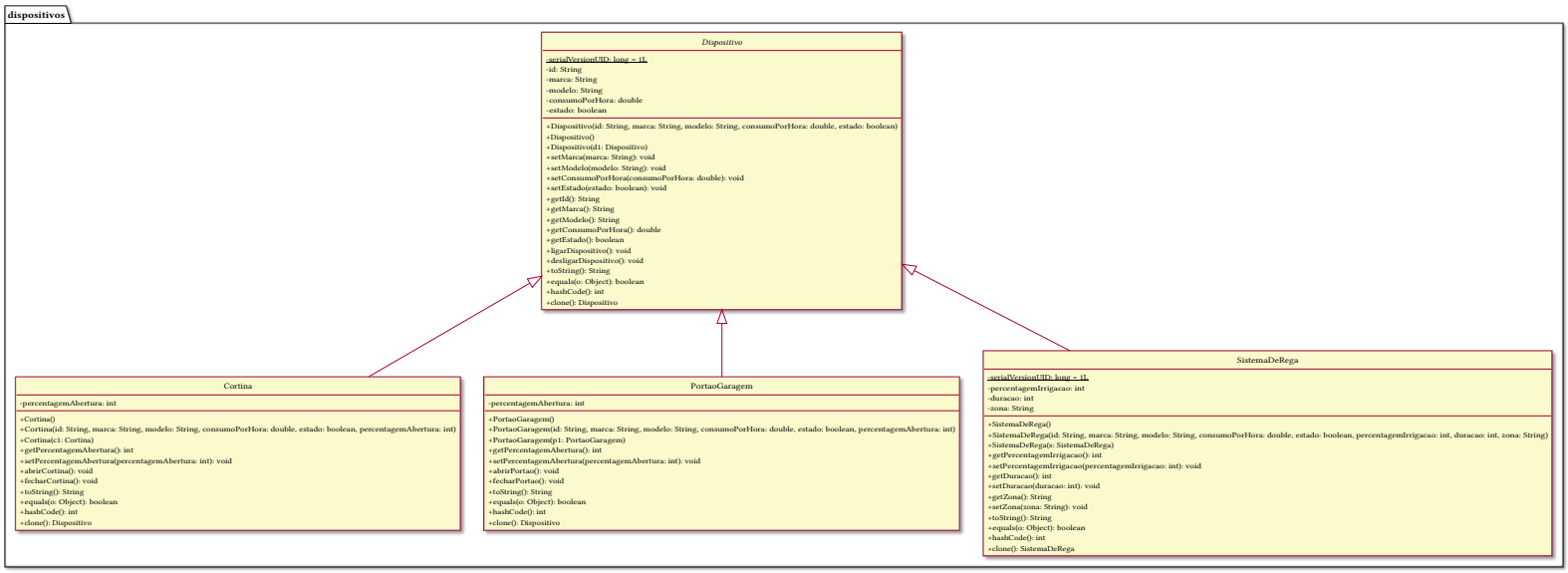


Figura 8: Diagrama de classes - Dispositivos (Parte 2)

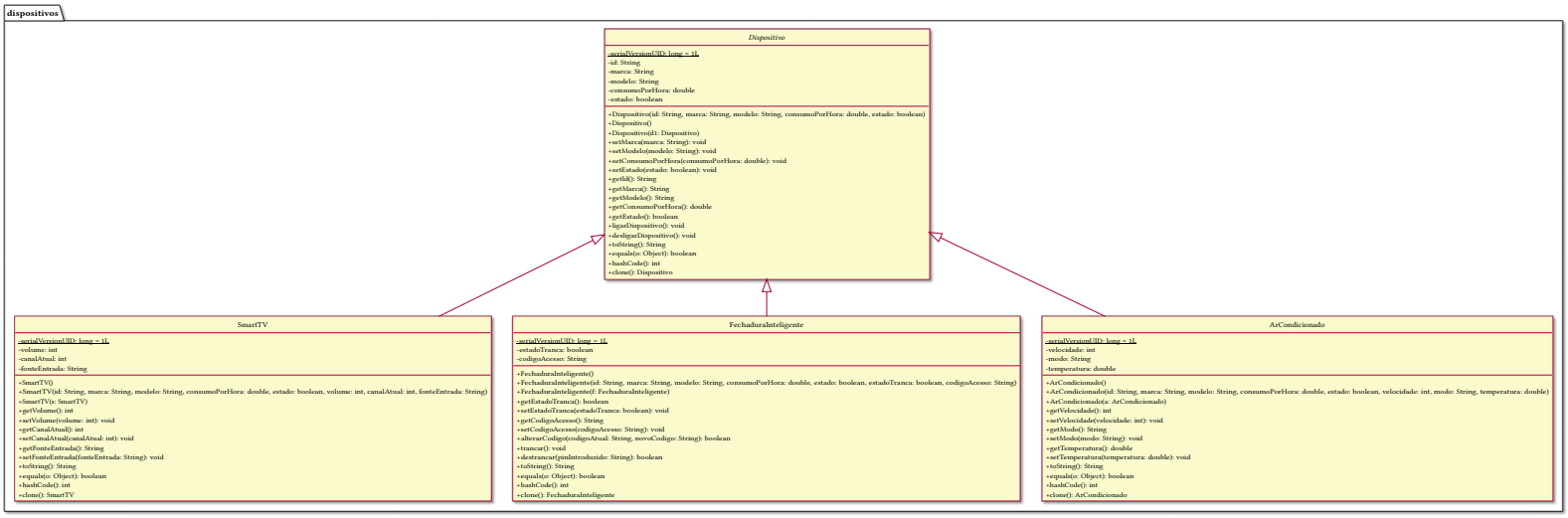


Figura 9: Diagrama de classes - Dispositivos (Parte 3)

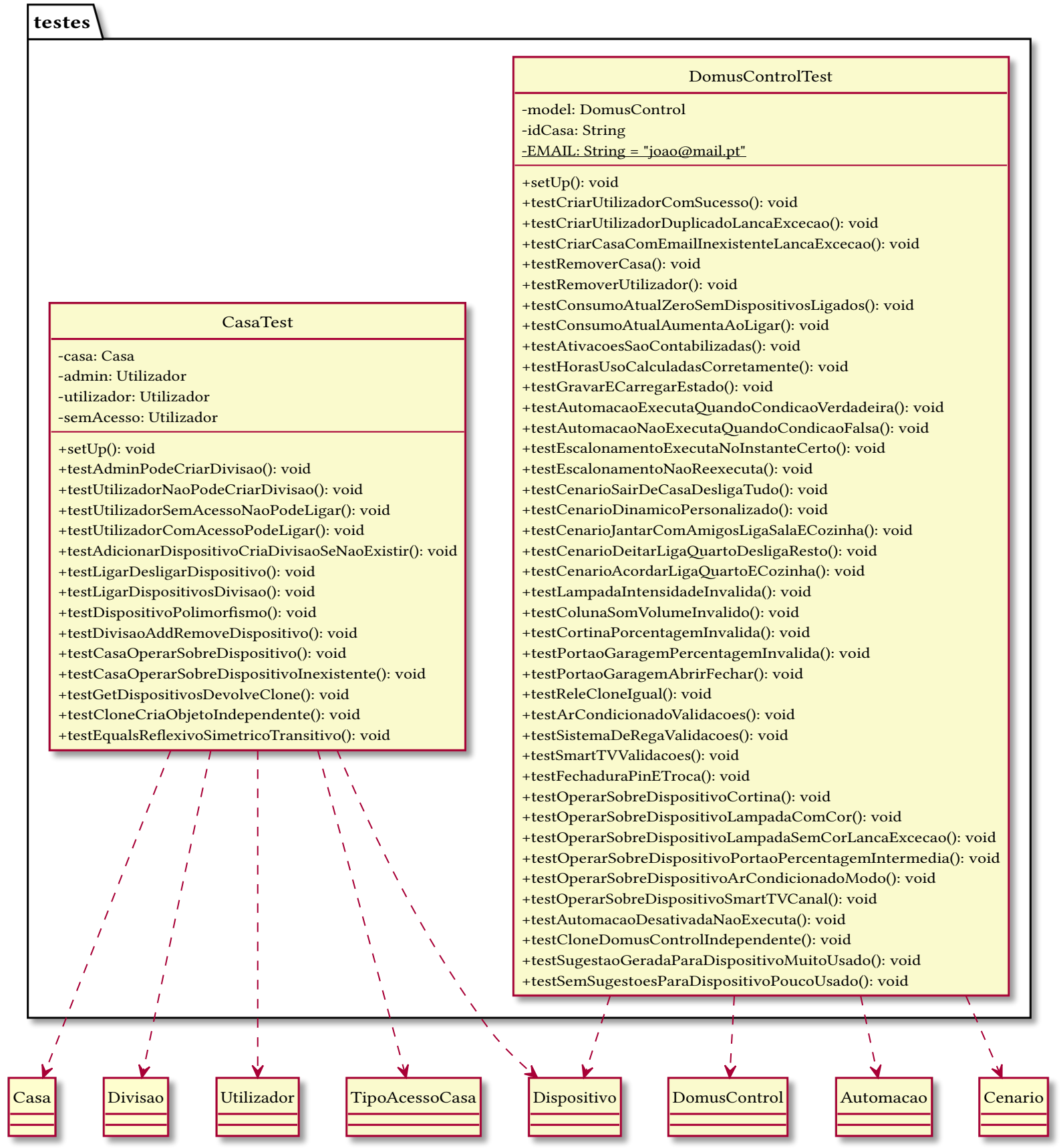


Figura 10: Diagrama de classes - Testes